

Reviewer Pack — Minimal Rank of Quaternionic Forms Bounds Circuit Depth of X...

Entry f18019fc9eff · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 10:36:41 UTC

Minimal Rank of Quaternionic Forms Bounds Circuit Depth of XOR 3-CNF

Entry ID: f18019fc9eff **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 10:36:41 UTC

1. Conjecture

Field A (mathematical branch): Quaternionic Geometry (Forms) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

[‘For every XOR 3-CNF circuit C with n variables, the minimal rank of its associated quaternionic form is $\Theta(\log(n))$.’, ‘Equivalently, for any circuit C computing XOR on n inputs, there exists a quaternionic form φ such that the rank of φ is at least $c \cdot \log(n)$ for some constant c .’, ‘No such bound holds for circuits computing AND or OR.’]

Rationale (proposer’s reasoning):

[‘Quaternionic geometry has been explored in algebraic geometry but rarely applied to circuit complexity. This conjecture aims to exploit quaternionic structures, which are non-commutative extensions of complex numbers, to derive lower bounds on circuit complexity.’, ‘The minimal rank of a quaternionic form could reveal inherent complexities in the structure of XOR 3-CNF circuits that are not apparent with traditional methods.’, ‘If true, this conjecture would provide new insights into the relationship between algebraic structures and computational complexity.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ec2e3b08cbb1b130

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 independent seeds, the average rank of quaternionic forms for XOR 3-CNF circuits with n variables ($n=1$ to 40) is at least $0.5 \cdot \log(n) + \epsilon$ for some $\epsilon > 0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank" quaternionic geometry [AND] XOR 3-CNF circuit complexity" - "quaternionic form" [AND] Boolean circuit depth" - " $\Theta(\log(n))$ " [AND] associated quaternionic forms XOR 3-CNF

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        if A[i][i] == 0:
            continue
        for j in range(n):
            A[i][j] /= A[i][i]
        for j in range(m):
            if j != i and A[j][i] != 0:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]

def rank_of_matrix(A):
    m, n = len(A), len(A[0])
    A_copy = [row[:] for row in A]
    gaussian_elimination(A_copy)
    rank = 0
    for i in range(m):
        if any(A_copy[i][j] != 0 for j in range(n)):
            rank += 1
```

```

    return rank

def construct_quaternionic_form(n):
    truth_table = [[random.choice([0, 1]) for _ in range(2**n)] for _ in range(2**n)]
    form = []
    for i in range(2**n):
        row = [truth_table[i][j] * (-1)**sum(truth_table[i][k] & truth_table[j][k] for k in range(2**n)) for j in range(2**n)]
        form.append(row)
    return form

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0

    for n in n_values:
        form = construct_quaternionic_form(n)
        rank = rank_of_matrix(form)
        total_rank += rank
        instances_tested += len(form)

    average_rank = Fraction(total_rank, instances_tested)
    conjecture_holds = average_rank >= 0.5 * math.log(instances_tested) + 1e-6
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "average_rank",
        "metric_value": float(average_rank),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30*31, 2))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{trial_result}...}}")
        results.append(trial_result)

    total_rank = sum(result["metric_value"] * result["instances_tested"] for result in results)
    instances_tested = sum(result["instances_tested"] for result in results)
    average_rank = Fraction(total_rank, instances_tested)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={average_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={average_rank} std=0.0 support_fraction={support_fraction}")
    else:
        for result in results:
            if not result["conjecture_holds"]:

```

```

        counterexample = result["counterexample"]
        first_failing_seed = result["seed"]
        break
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm whether the average rank of quaternionic forms meets the support condition. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11983
2	preregistration	ollama_remote	glm4:latest	0	0	5904
3	novelty	ollama_remote	glm4:latest	0	0	4921
4	novelty	ollama_remote	glm4:latest	0	0	5747
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40531
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11600
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15481
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10791
9	judge	ollama_remote	glm4:latest	0	0	16449

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 123408 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/f18019fc9eff.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/f18019fc9eff.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f18019fc9eff.tar.gz` (if generated)